

Bucles y declaraciones condicionales

Santiago Gallego

2026-05-07

Contenido

0.1	Ejercicio 1	2
0.1.1	python	2
0.1.2	R	2
0.1.3	python	3
0.1.4	R	3
0.1.5	python	3
0.1.6	R	3
0.1.7	python	4
0.1.8	R	4
0.2	Ejercicio 2	5
0.2.1	python	5
0.2.2	R	6
0.2.3	python	6
0.2.4	R	6
0.2.5	python	7
0.2.6	R	7
0.3	Preguntas	8

0.1 Ejercicio 1

Contexto: Estás trabajando en un proyecto de zonificación agrícola en el Eje Cafetero. Tienes una lista de fincas con su respectiva altitud. Sabiendo que el café de especialidad de la región requiere alturas entre los 1700 y los 2200 metros sobre el nivel del mar (msnm), necesitas crear un algoritmo que clasifique automáticamente cada finca.

0.1.1 python

```
# 1. Definición de datos: creamos la lista con las alturas de las fincas
altitudes_fincas = [1350, 1750, 1900, 2300, 1600, 2100]
```

0.1.2 R

```
# 1. Definición de datos: creamos el vector con las altitudes
altitudes_fincas <- c(1350, 1750, 1900, 2300, 1600, 2100)
```

2. Inicializa dos contadores en cero: conteo_especial y conteo_tradicional.

3. Inicializa dos colecciones vacías (listas o vectores) llamadas `fincas_especiales` y `fincas_tradicionales`.

0.1.3 python

```
# 2 y 3. Inicialización de contadores (enteros) y listas vacías para clasificar
conteo_especial = 0
conteo_tradicional = 0
fincas_especiales = []
fincas_tradicionales = []
```

0.1.4 R

```
# 2 y 3. Inicialización de contadores y vectores vacíos para el almacenamiento
conteo_especial <- 0
conteo_tradicional <- 0
fincas_especiales <- c()
fincas_tradicionales <- c()
```

4. Construye un ciclo for que recorra uno a uno los valores de `altitudes_fincas`.
5. Dentro del ciclo, utiliza una declaración if - else con operadores lógicos (AND) para evaluar cada altitud:
 - Si la altitud es mayor o igual a 1700 Y menor o igual a 2200, agrega la altitud a la colección `fincas_especiales` y suma 1 al `conteo_especial`.
 - De lo contrario, agrégala a la colección `fincas_tradicionales` y suma 1 al `conteo_tradicional`.

0.1.5 python

```
# 4. Ciclo for: recorreremos cada valor dentro de la lista de altitudes
for altitud in altitudes_fincas:
    # 5. Lógica de clasificación: evaluamos si está en el rango óptimo (1700-2200)
    if altitud >= 1700 and altitud <= 2200:
        fincas_especiales.append(altitud) # Agregamos el valor a la lista de especiales
        conteo_especial += 1               # Incrementamos el contador respectivo
    else:
        fincas_tradicionales.append(altitud) # Agregamos a la lista de tradicionales
        conteo_tradicional += 1              # Incrementamos el contador respectivo
```

0.1.6 R

```
# 4. Ciclo for: iteramos sobre cada elemento del vector de altitudes
for (altitud in altitudes_fincas) {
  # 5. Lógica de clasificación: usamos operadores lógicos para validar el rango
  if (altitud >= 1700 && altitud <= 2200) {
    fincas_especiales <- c(fincas_especiales, altitud) # Concatenamos el valor al vector
    conteo_especial <- conteo_especial + 1             # Aumentamos el contador
  } else {
    fincas_tradicionales <- c(fincas_tradicionales, altitud) # Concatenamos el valor
    conteo_tradicional <- conteo_tradicional + 1           # Aumentamos el contador
  }
}
```

6. Fuera del ciclo, imprime un reporte final que muestre cuántas fincas son aptas para café de especialidad y cuántas para café tradicional, mostrando además los valores guardados en ambas colecciones.

0.1.7 python

```
# 6. Reporte final: mostramos los resultados usando f-strings para interpolar variables
print("--- REPORTE DE ZONIFICACIÓN CAFETERA ---")
```

```
--- REPORTE DE ZONIFICACIÓN CAFETERA ---
```

```
print(f"Fincas Especiales: {conteo_especial} (Altitudes: {fincas_especiales})")
```

```
Fincas Especiales: 3 (Altitudes: [1750, 1900, 2100])
```

```
print(f"Fincas Tradicionales: {conteo_tradicional} (Altitudes: {fincas_tradicionales})")
```

```
Fincas Tradicionales: 3 (Altitudes: [1350, 2300, 1600])
```

0.1.8 R

```
# 6. Reporte final: imprimimos el resumen del proceso con cat y paste
cat("--- REPORTE DE ZONIFICACIÓN CAFETERA ---\n")
```

```
--- REPORTE DE ZONIFICACIÓN CAFETERA ---
```

```
cat(paste("Fincas Especiales:", conteo_especial, "\n"))
```

```
Fincas Especiales: 3
```

```
cat("Altitudes:", paste(fincas_especiales, collapse = ", "), "\n\n")
```

Altitudes: 1750, 1900, 2100

```
cat(paste("Fincas Tradicionales:", conteo_tradicional, "\n"))
```

Fincas Tradicionales: 3

```
cat("Altitudes:", paste(fincas_tradicionales, collapse = ", "), "\n")
```

Altitudes: 1350, 2300, 1600

0.2 Ejercicio 2

Contexto: El IDEAM te ha encargado programar el software de una estación hidrológica automatizada en el Río Magdalena. El sensor debe tomar lecturas continuas del nivel del agua. Si el río supera los 8.5 metros, el sistema debe disparar una alerta de inundación y detener las mediciones para proteger el equipo. Si después de cierto número de lecturas todo está normal, el sensor entra en modo de reposo.

1. Define las siguientes variables de configuración:

nivel_alerta = 8.5 max_lecturas = 12 lectura_actual = 0 Una bandera lógica (Booleana) llamada alerta_inundacion inicializada como Falsa (False / FALSE / false).

0.2.1 python

```
import random # Importamos el módulo para generar números aleatorios

# 1. Configuración inicial: definimos umbrales, límites y estado inicial
nivel_alerta = 8.5
max_lecturas = 12
lectura_actual = 0
alerta_inundacion = False # Bandera booleana para rastrear si hubo emergencia
```

0.2.2 R

```
# 1. Configuración inicial: asignamos valores base para el monitoreo
nivel_alerta <- 8.5
max_lecturas <- 12
lectura_actual <- 0
alerta_inundacion <- FALSE # Bandera lógica inicializada en falso
```

2. Construye un ciclo while que se ejecute mientras lectura_actual sea menor a max_lecturas.
3. Dentro del ciclo, simula la medición del río generando un número decimal aleatorio entre 5.0 y 9.0 (usa la función nativa de tu lenguaje para números aleatorios, ej. random.uniform en Python, runif en R, o rand en Julia).
4. Incrementa en 1 la variable lectura_actual. Imprime en pantalla el número del intento y el nivel del agua medido (con dos decimales).

0.2.3 python

```
# 2. Ciclo de monitoreo: se ejecuta mientras no superemos el máximo de intentos
while lectura_actual < max_lecturas:
    # 3. Simular lectura: generamos nivel aleatorio e incrementamos el contador
    nivel_agua = random.uniform(5.0, 9.0)
    lectura_actual += 1

    # Imprimimos el resultado del sensor con 2 decimales
    print(f"Intento {lectura_actual}: Nivel del agua = {nivel_agua:.2f} m")

    # 4. Condicional de alerta: si se supera el nivel, activamos bandera y salimos
    if nivel_agua >= nivel_alerta:
        alerta_inundacion = True # Cambiamos el estado de la bandera a Verdadero
        print("¡ALERTA ROJA! Posible desbordamiento.")
        break # Rompemos el ciclo inmediatamente para proteger el sensor
```

```
Intento 1: Nivel del agua = 5.42 m
Intento 2: Nivel del agua = 5.02 m
Intento 3: Nivel del agua = 7.48 m
Intento 4: Nivel del agua = 8.86 m
¡ALERTA ROJA! Posible desbordamiento.
```

0.2.4 R

```
# 2. Ciclo de monitoreo: estructura while para lecturas continuas
while (lectura_actual < max_lecturas) {
    # 3. Simular lectura: generamos 1 valor aleatorio entre 5 y 9
```

```

nivel_agua <- runif(1, min = 5.0, max = 9.0)
lectura_actual <- lectura_actual + 1

# Mostramos el intento actual formateado
cat(sprintf("Intento %d: Nivel del agua = %.2f m\n", lectura_actual, nivel_agua))

# 4. Condicional de alerta: validamos el peligro y detenemos el proceso si ocurre
if (nivel_agua >= nivel_alerta) {
  alerta_inundacion <- TRUE # Actualizamos la bandera a VERDADERO
  cat("¡ALERTA ROJA! Posible desbordamiento.\n")
  break # Salimos del ciclo break inmediatamente
}
}

```

```

Intento 1: Nivel del agua = 5.70 m
Intento 2: Nivel del agua = 5.15 m
Intento 3: Nivel del agua = 5.03 m
Intento 4: Nivel del agua = 6.26 m
Intento 5: Nivel del agua = 6.19 m
Intento 6: Nivel del agua = 5.83 m
Intento 7: Nivel del agua = 7.18 m
Intento 8: Nivel del agua = 6.05 m
Intento 9: Nivel del agua = 7.78 m
Intento 10: Nivel del agua = 8.30 m
Intento 11: Nivel del agua = 6.17 m
Intento 12: Nivel del agua = 6.84 m

```

5. Usa un condicional if para verificar si la medición superó o igualó el nivel_alerta. Si es así:
6. Cambia la bandera alerta_inundacion a Verdadera. Imprime un mensaje de “¡ALERTA ROJA! Posible desbordamiento.” Rompe el ciclo inmediatamente usando el comando adecuado (break). Fuera del ciclo while, utiliza un condicional para evaluar tu bandera lógica. Si la bandera sigue siendo Falsa, imprime: “Monitoreo finalizado. Niveles estables.”

0.2.5 python

```

# 5 y 6. Evaluación final: si la bandera sigue falsa, el monitoreo fue exitoso
if not alerta_inundacion:
    print("Monitoreo finalizado. Niveles estables.")

```

0.2.6 R

```

# 5 y 6. Evaluación final: verificamos si se activó la alerta durante el ciclo
if (!alerta_inundacion) {
  cat("Monitoreo finalizado. Niveles estables.\n")
}

```

Monitoreo finalizado. Niveles estables.

0.3 Preguntas

1. Indentación vs. Delimitadores (Python vs. R/Julia) Esta es la diferencia estética y funcional más famosa entre estos lenguajes. En Python: El computador utiliza la indentación (espacios en blanco). No existen corchetes ni palabras clave como `end`. Si una línea está movida 4 espacios a la derecha bajo un `for`, Python asume que es parte del ciclo. Cuando la línea vuelve al margen izquierdo, el ciclo ha terminado. Es un sistema “obligatorio” que fuerza al programador a ser ordenado. En R y Julia: La estructura se define por delimitadores explícitos. En R, se usan llaves `{ }`. Todo lo que esté entre las llaves pertenece al bloque, sin importar cuántos espacios uses (aunque se recomienda indentar por legibilidad). En Julia, se usa la palabra clave `end` para cerrar cada bloque (`if`, `for`, `while`).
2. El peligro del Ciclo Infinito, si se olvidas a instrucción `lectura_actual += 1` en un ciclo `while`: En la Memoria y Ejecución: El programa entraría en un bucle infinito. La condición `lectura_actual < max_lecturas` siempre sería verdadera (ej. `0 < 12`). El procesador se quedaría atrapado ejecutando la misma tarea al 100% de su capacidad, el sensor generaría millones de lecturas por segundo y, dependiendo de cómo se guarden los datos, podrías agotar la memoria RAM o llenar el disco duro con logs en cuestión de minutos. ¿Por qué no pasa en el `for`? El ciclo `for` es iterativo sobre una colección. El `for` toma una lista (como las 6 altitudes) y el lenguaje se encarga automáticamente de pasar al siguiente elemento. No depende de que uno mueva un contador; el ciclo “sabe” que cuando llega al final de la lista, debe detenerse.
3. Operadores Lógicos: El doble (`&&`) frente al sencillo (`&`) Este es un punto crítico en R y Julia, lenguajes diseñados para el cálculo matricial y vectorial. El operador doble (`&&` / `||`): Se llama operador “Short-circuit” y está diseñado para comparar un solo valor booleano contra otro. Si le pasas un vector de 100 altitudes, el `&&` solo mirará el primer elemento del vector, ignorando el resto y devolviendo un solo resultado. En un filtro de zonificación, esto ignoraría 99 fincas. El operador sencillo (`&` / `.` en Julia): Se llama operador vectorial (element-wise). Este operador compara cada posición del vector una por una. En R: `altitudes > 1700 & altitudes < 2200` genera un vector de `TRUE` y `FALSE` del mismo tamaño que el original. En Julia: Se usa el punto `.` para indicar explícitamente que la operación debe “esparcirse” (broadcast) sobre todos los elementos del arreglo. Regla de oro: Usa `&&` dentro de un `if` (donde solo evalúas una condición a la vez) y usa `&` cuando se esta filtrando tablas, vectores o matrices de datos.